

Natural Language Processing

13. Interpretability

Juan José Alegría

June 17, 2025

Disclaimer

1. Although this is an NLP course, in this lesson we will take a broader view and explore interpretability in machine learning more generally. We will draw connections to text data when relevant, and return our focus to NLP-specific topics toward the end of the lesson.
2. Much of the material in this lesson is based on Christoph Molnar's book *Interpretable Machine Learning* [Molnar, 2025]. For the section on SHAP, I also draw from the Medium article *From Shapley to SHAP — Understanding the Math* by Conor O'Sullivan [O'Sullivan, 2023].

1. Interpretability: definition and fundamentals

2. Interpretability methods

Interpretability: definition and fundamentals

Clever Hans



Figure 1: Clever Hans, the mathematical horse.

Loan Approval Prediction

- Suppose you work for a bank and are tasked with developing a machine learning model to predict whether a loan should be approved for a client, based on their financial history.
- You gather the data, run all the experiments, and end up with a strong (though complex) model. It performs well on the test data.
- The model works so well that it gets deployed and is used to decide whether client applications should be approved or denied. The bank is happy — everything seems fine.
- One day, the model denies a client's application, and the client asks why. The bank clerk can't just say "the machine said so," so they turn to you — the machine learning engineer — for an explanation.
- What do you do?

What's interpretability?

- *“Interpretability is the degree to which a human can understand the cause of a decision.”* [Biran and Cotton, 2017]
- *“A method is interpretable if a user can correctly and efficiently predict the method’s results”* [Kim et al., 2016]
- *“[Interpretability is] the ability to explain or to present in understandable terms to a human”* [Doshi-Velez and Kim, 2017]

Why interpretability?

- *“The need for interpretability stems from an incompleteness in the problem formalization, creating a fundamental barrier to optimization and evaluation.”*
[Doshi-Velez and Kim, 2017]
- In other words, when we build a model for prediction or classification, we may be interested in more than just its performance. In particular, we may want to:
 - **Gain knowledge:** Understand the relationship between the target variable and the input features.
 - **Ensure safety:** For example, in self-driving cars, we need to verify that the model won't fail under unexpected conditions.
 - **Eliminate biases:** Ensure that the model is not relying on or amplifying racial, gender, or social biases.
- In all these cases, interpretability can help by providing insights into how the model makes its predictions.

Taxonomy of Interpretability

There are many ways to classify interpretability methods. In this discussion, we follow the dimensions proposed by [Madsen et al., 2022]:

- **Scope:** *What is the scope of the explanation?* It can be *local* (explains a single observation) or *global* (summarizes the overall behavior of the model).
- **Time:** *When is the explanation generated?* We distinguish between *post-hoc methods*, where the explanation is generated after the model has made a prediction, and *intrinsic* (or *self-explanatory*) methods, which use models that are interpretable by design.
- **Input:** *What does the interpretability method require from the model?* It may treat the model as a *black box* (accessing only input-output pairs), or it may require access to the training data, gradients, embeddings, or full *white-box* access to the model's internal mechanisms.
- **Output:** *What is the format of the explanation?* For local methods, outputs can include saliency scores, natural language explanations, counterfactuals, or adversarial examples.

What is a good explanation?

- Borrowing from psychology, explanations can be defined as “the currency in which we exchange beliefs” [Lombrozo, 2006].
- Besides a formal definition, it is also difficult to determine what constitutes a good explanation. After reviewing philosophical, cognitive, and social literature, Miller [Miller, 2019] found that good explanations are: i) *contrastive* —people don’t ask why an event P happened, but why P happened instead of another event Q —, *selected* —people want one or two causes for an event, instead of the whole list of reasons—, and *social* —they are a “transfer of knowledge”, involving prior beliefs from the explainer and the explainee—.
- In ML, the literature focuses on two main desired properties [Jacovi and Goldberg, 2020]:
 - **Plausibility**: how convincing the interpretation is to humans.
 - **Faithfulness**: how accurately the explanation reflects the true reasoning of the model.
- An explanation can be *faithful, but implausible*, and it can also be *plausible, but unfaithful*.

Interpretability methods

Self-explanatory methods

- Based on models that are considered interpretable by design.
- For instance, in logistic regression, you can inspect the weights learned by the model.
- In a decision tree, you can follow the path that leads to a particular decision.
- There are limitations: logistic regression and decision trees are not as powerful as more advanced models.

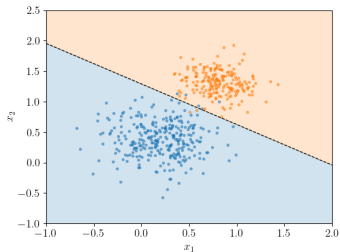


Figure 2: Logistic Regression. Source: SciPython.com

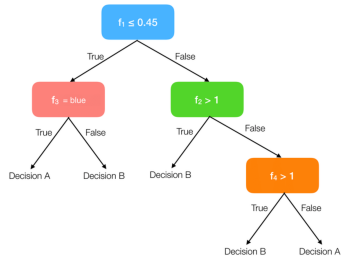


Figure 3: Decision Tree. Source: ResearchGate

Partial Dependence Plots

- **Idea:** Plot how a feature (or a set of features) affects the output of a machine learning model (e.g., the predicted probability for a given class).
- Let X_S be the set of features of interest, and X_C its complement (i.e., the remaining features). Thus, $X = X_S \cup X_C$.
- Let f be the trained model that maps feature vectors $x = (x_S, x_C)$ to predictions.
- The partial dependence function for X_S is defined as the expected value of the model's prediction when X_S is fixed and X_C varies according to its marginal distribution:

$$\begin{aligned}pd_{X_S}(x_S) &= \mathbb{E}_{X_C}[f(x_S, X_C)] \\ &= \int f(x_S, x_C) p(x_C) dx_C\end{aligned}$$

- In practice, this expectation is approximated using the empirical distribution of the dataset:

$$\widehat{pd}_{X_S}(x_S) = \frac{1}{n} \sum_{i=1}^n f(x_S, x_C^{(i)})$$

Partial Dependence Plots

- In other words, for a fixed value x_S of the feature(s) of interest, we modify each instance in the dataset by replacing its X_S features with x_S , while keeping the original values of X_C (i.e., $x_C^{(i)}$). We then compute $f(x_S, x_C^{(i)})$ for each modified instance, average the results, and obtain the partial dependence value. Repeating this for different values of x_S gives the PDP curve.
- This method is *global* (it aims to explain the overall behavior of the model), *post-hoc* (it generates explanations after the model has been trained), *model-agnostic* (it only requires access to the model's inputs and outputs, as in a black-box), and produces an output in the form of a plot.

Partial Dependence Plot

In Python, you can use `PartialDependenceDisplay`, provided by `sklearn.inspection`, to compute and visualize PDPs.

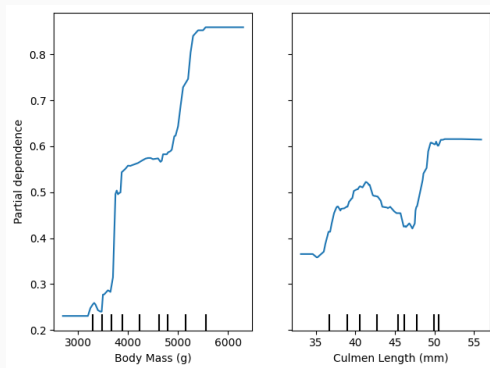


Figure 4: PDP for features “Body Mass” and “Culmen Length” in the task of classifying penguins according to their gender.

- **Local Interpretable Model-agnostic Explanations (LIME)**, proposed by [Ribeiro et al., 2016].
- **Idea:** train a local **surrogate model** that is interpretable to approximate the predictions of the black box model in a neighborhood.
- Recipe:
 - Select the data point for which you want to have an explanation of its black box prediction.
 - Perturb the dataset and get the black box predictions for these new points.
 - Weight the new samples according to their proximity to the instance of interest.
 - Train a weighted, interpretable model on the dataset with the variations; for example, LASSO or a Decision Tree.
 - Explain the prediction by interpreting the local model.

Mathematically:

$$\text{explanation}(x) = \arg \min_{g \in G} L(\hat{f}, g, \pi_x) + \Omega(g)$$

L = loss (e.g., mean squared error)

$\hat{f}$ = original black-box model (e.g., xgboost, neural network, etc)

G = family of possible explanations (e.g., all possible linear regression models)

π_x = proximity measure, which defines how large is the neighborhood around x

$\Omega(g)$ = model complexity (number of features to use in the explanation)

g = function $\in G$ that minimizes the loss and that is provided as explanation

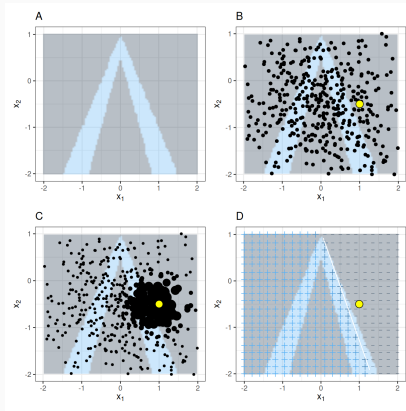


Figure 5: LIME algorithm for tabular data. A) Prediction surface given features x_1 and x_2 . Predicted classes: 1 (dark) or 0 (light). B) Instance of interest (big dot) and sampled data (small dots). C) Assign weights based on distance to instance. D) Signs (+/-) show the classifications of the locally learned model from the weighted samples. The white line marks the decision boundary ($P(c = 1) = 0.5$)

The choice of neighborhood is really important.

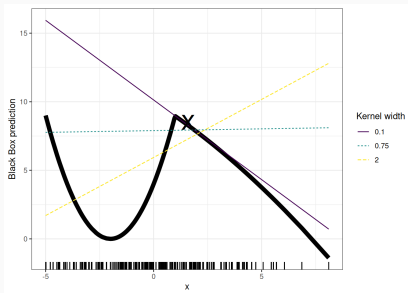


Figure 6: Explanation of the prediction of instance $x = 1.6$ with different kernel widths. The model predictions are shown as a thick line and the distribution of the data is shown with rugs. Three local surrogate models with different kernel widths are computed.

LIME for text data

- The method to generate perturbations in text data is ad-hoc: starting from the original text, new texts are created by randomly removing words from the original text.
- The dataset is represented with binary features for each word. A feature is 1 if the corresponding word is included and 0 if it has been removed.
- Suppose that we have a dataset with YouTube comments, tagged as "spam" (1) or "no spam" (0).
- We trained a black-box model to classify the comments. It could be a random forest, xgboost, neural network, etc.

	CONTENT	CLASS
267	PSY is a good guy	0
173	For Christmas Song visit my channel! ;)	1

Figure 7: Two examples of comments.

LIME for text data

- The next step is to create variations of the points of the dataset.
- To do that, we delete words from each sentence.

For	Christmas	Song	visit	my	channel!	;)	prob	weight
1	0	1	1	0	0	1	0.17	0.57
0	1	1	1	1	0	1	0.17	0.71
1	0	0	1	1	1	1	0.99	0.71
1	0	1	1	1	1	1	0.99	0.86
0	1	1	1	0	0	1	0.17	0.57

Figure 8: Variations of one of the YouTube comments along with their weights (based on distance) and predicted probability of being spam. 1 indicates that the word is present, 0 otherwise; for instance, the first row would be "For Song visit :)". The weight is computed as $1 - \frac{n_{\text{deleted}}}{n_{\text{total}}}$.

LIME for text data

- Train a surrogate interpretable model (e.g., logistic regression) and interpret the results.

case	label_prob	feature	feature_weight
1	0.1701170	is	0.000000
1	0.1701170	good	0.000000
1	0.1701170	a	0.000000
2	0.9939024	channel!	6.180747
2	0.9939024	;))	0.000000
2	0.9939024	visit	0.000000

Figure 9: LIME explanations for text classification. The word *channel* indicates a high probability of spam. For the non-spam comment, no non-zero weight was estimated, because no matter which word is removed, the predicted class remains the same.

LIME is:

- **Local:** explains a single prediction.
- **Post-hoc:** the explanation is generated after the model has been trained and the prediction generated.
- **Model-agnostic:** requires only access to inputs and outputs (black-box), not to the internal workings of the model.
- **Output:** returns importance scores for each input feature, indicating how much each contributed to the specific prediction.

- **SH**apley **Additive** ex**Plan**ations (SHAP), proposed by [Lundberg and Lee, 2017].
- Builds on the concept of Shapley values from cooperative game theory [Shapley and others, 1953].
- Prior to SHAP, Shapley values had already been proposed to estimate feature importance, but SHAP introduced efficient algorithms for computing them, along with a widely used library: <https://github.com/shap/shap>.
- So, what are Shapley values?

Shapley values

- Shapley values are a method for **fairly** distributing the total gains (or costs) among a group of players who have collaborated.
- But how does this idea work in practice?
- Imagine that you and a friend entered a contest and together won the first prize: \$10,000.
- Now, you want to split the money. An equal split is one option, but you believe you contributed more and should receive a larger share.
- So how can you divide the money in a way that is considered fair by both sides?

Shapley values

- Suppose you also have a time machine and can re-enter the contest alone — and your friend can do the same.
- When you compete alone, you win second place and earn \$7,500.
- Your friend competes alone and earns third place, receiving \$5,000.
- Naturally, if neither of you enters the contest, you both earn \$0.
- We can record the value of each of these *coalitions* — groups of players working together — as the basis for calculating a fair distribution.

$$\begin{aligned}C_{12} &= 10,000 \\C_1 &= 7,500 \\C_2 &= 5,000 \\C_0 &= 0\end{aligned}$$

Figure 10: Values for each coalition. 0 → no players. 1 → you. 2 → your friend. 12 → you and your friend

Shapley values

- To compute how much each player should receive, we calculate their **expected marginal contribution**: the weighted average of a player's added value to every coalition they could join.
- For example, Player 1 could join a coalition of only Player 2. That coalition improves from third place (\$5,000) to first place (\$10,000), so Player 1 contributes $\$10,000 - \$5,000 = \$5,000$.
- Player 1 could also join a coalition of no one. In that case, they go from \$0 to \$7,500, contributing \$7,500.
- Averaging these values gives Player 1 an expected marginal contribution of \$6,250.

$$C_{12} - C_2 = 5,000$$

$$C_1 - C_0 = 7,500$$

$$(\$5,000 + \$7,500) / 2 = \$6,250$$

Figure 11: Expected marginal contribution for Player 1.

Shapley values

- We can follow the same process to compute the expected marginal contribution of Player 2.
- This time, the value turns out to be **\$3,750**.
- In the end, Player 1 receives \$6,250 and Player 2 receives \$3,750.
- These are the **Shapley values** for Player 1 and Player 2 — a fair distribution based on their individual contributions.
- Notice that the Shapley values add up to the total prize of \$10,000, preserving the total value.

$$\begin{aligned}C_{12} - C_2 &= 5,000 \\ C_{1} - C_0 &= 7,500\end{aligned}$$

$$(5,000 + 7,500) / 2 = \mathbf{\$6,250}$$

Figure 12: Expected marginal contribution for Player 1.

Shapley values

- Let's move to a slightly more complex example, using three players.
- Now there are 8 possible coalitions: 1 with 3 players (C_{123}), 3 with 2 players (C_{12} , C_{13} , C_{23}), 3 with 1 player (C_1 , C_2 , C_3) and 1 with no players (C_0).

$C_{123} = 10,000$	$C_{12} = 7,500$	$C_1 = 5,000$
$C_0 = 0$	$C_{13} = 7,500$	$C_2 = 5,000$
	$C_{23} = 5,000$	$C_3 = 0$

Figure 13: Values for each coalition with three possible players.

Shapley values

- Let's calculate the Shapley value for Player 1 (P1).
- With three players, there are now four coalitions that P1 could join: i) a coalition with both P2 and P3, ii) a coalition with only P2, iii) a coalition with only P3, and iv) or a coalition with no one.
- As before, we calculate the **marginal contribution** of P1 to each of these coalitions — that is, how much additional value P1 brings when joining each group.
- Finally, we compute the **weighted average** of these contributions.
- This gives us a Shapley value of \$5,000.

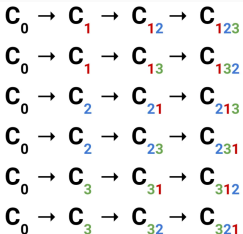
$$\begin{array}{rcll} C_{123} - C_{23} & = & 5,000 & \\ C_{12} - C_2 & = & 2,500 & \\ C_{13} - C_3 & = & 7,500 & \\ C_1 - C_0 & = & 5,000 & \end{array} \quad \begin{array}{l} 5,000 * (\frac{1}{3}) + 2,500 * (\frac{1}{6}) \\ + 7,500 * (\frac{1}{6}) + 5,000 * (\frac{1}{3}) \\ = \text{\$5,000} \end{array}$$

Figure 14: Expected marginal contribution of Player 1 on the game with three players.

Shapley values

- Where do these weights come from?
- They represent the **probabilities** that P1 makes a specific marginal contribution when joining a coalition. Weighting by these probabilities gives us the **expected marginal contribution**.
- To calculate them, we consider all the possible orders in which the players can join the coalition.
- With 3 players, there are $3! = 6$ possible permutations (e.g., P1–P2–P3, P2–P3–P1, etc.).
- In each permutation, we look at the set of players who arrive before P1 — that is the coalition P1 is joining.
- For each of these coalitions, we compute P1's marginal contribution and then average them, weighting each by how frequently the coalition appears across all permutations.

Shapley values



- When P1 joins the coalition of P2 and P3, their marginal contribution is \$5,000. This happens in two coalitions: C_{231} and C_{321} . So the probability is $\frac{2}{6} = \frac{1}{3}$.
- When P1 joins a coalition of only P2, the contribution is \$2,500. This occurs in one coalition: C_{213} . So the probability is $\frac{1}{6}$.
- When P1 joins a coalition of only P3, the contribution is \$7,500. This happens in one coalition: C_{312} . So the probability is $\frac{1}{6}$.
- When P1 joins a coalition of no players (i.e., P1 is first), the contribution is \$5,000. This occurs in two coalitions: C_{123} and C_{132} . So the probability is $\frac{2}{6} = \frac{1}{3}$.
- Using this procedure, the Shapley value for P1 is \$5,000, for P2 is \$3,750, and for P3 is \$1,250. Again, the Shapley values sum to the total prize money of \$10,000.

Shapley values

$$\phi_i = \sum_{S \subseteq \{1, \dots, p\}, i \notin S} \underbrace{\frac{|S|!(p-|S|-1)!}{p!}}_{\text{Weight}} \underbrace{[val(S \cup \{i\}) - val(S)]}_{\text{Marginal contribution of player } i \text{ to coalition } S}$$

Figure 15: Shapley value for player i of a p player game

- This is the general formula of Shapley values.
- We are summing over all coalitions S that don't include the player i . I.e., all the coalitions where the player i can make a marginal contribution.
- The weight is given by $\frac{|S|!(p-|S|-1)!}{p!}$, where:

$|S|$ = size of coalition S

p = number of players

$|S|!$ = number of ways players can join S before player i

$(p - |S| - 1)$ = number of ways players can join the coalition after player i joins

$p!$ = number of ways to form a coalition of p players

Shapley axioms

Shapley values are derived from three axioms that can be considered a definition of **fairness**:

- **Symmetry**: Two players are considered interchangeable if they make the same contributions to all coalitions. If two players are interchangeable then they must be given an equal share of the game's total value.
- **Null player property**: If a player makes zero marginal contribution to all coalitions then they get none of the total value.
- **Additivity**: If we combine two games, then a player's overall contribution is the sum of the contributions for the two individual games. This axiom makes the assumption that any games played are independent.

It can also be proved that the Shapley value is the only **efficient** value that satisfies these 3 axioms. By efficient, we mean none of the game's value is left over. Under this definition, it is proved that the Shapley value is the only **fair** way to divide the value among the players.

- Ok, but how do Shapley values relate to ML?
- We set the value of the game to be the **model prediction** and the **feature values** are the players.
- Notice, the feature values are the players, not the feature themselves.
- Then, going back to the formula, S is the subset of the features used by the model and p is the total number of features. Also, x is the vector of feature values of the instance to be explained.
- We need to compute $val_x(S)$, the prediction for feature values in set S that are marginalized over features X_C , which are all the features not included in S :

$$val_x(S) = \int \hat{f}(x_1, \dots, x_p) d\mathbb{P}_{X_C} - \mathbb{E}[f(\hat{X})]$$

- We must perform multiple integrations for each feature that is not contained in S . For instance, suppose a model that receives four inputs: X_1, X_2, X_3, X_4 . And we evaluate the prediction for the coalition S consisting of feature values x_1 and x_3 :

$$val_x(S) = \int_{\mathbb{R}} \int_{\mathbb{R}} \hat{f}(x_1, X_2, x_3, X_4) d\mathbb{P}_{X_2 X_4} - \mathbb{E}[\hat{f}(X)]$$

- All possible coalitions (sets) of feature values have to be evaluated with and without the j -th feature to calculate the exact Shapley value.
- For more than a few features, the exact solution to this problem becomes problematic as the number of possible coalitions exponentially increases as more features are added.
- A solution is to use an approximation with Monte Carlo sampling:

$$\hat{\phi}_j = \frac{1}{M} \sum_{m=1}^M \left(\hat{f}(x_{+j}^{(m)}) - \hat{f}(x_{-j}^{(m)}) \right)$$

- $\hat{f}(x_{+j}^{(m)})$ is the prediction for x , but with a random number of feature values replaced by feature values from a random data point z , except for the respective value of j .
- The feature vector $x_{-j}^{(m)}$ is almost identical to $x_{+j}^{(m)}$, but the value $x_j^{(m)}$ is also taken from the sampled z .
- Each of these M new instances is a kind of “Frankenstein’s Monster” assembled from two instances: the original data point we want to explain x and a random data point z .

Algorithm to approximate the Shapley value for a single feature value:

- Output: Shapley value for the value of the j -th feature
- Required: Number of iterations M , instance of interest x , feature index j , data matrix X , and machine learning model $\hat{f}$
- For all $m = 1, \dots, M$:
 - Draw random instance z from the data matrix X .
 - Choose a random permutation o of the feature values
 - Order instance x : $x_o = (x_{(1)}, \dots, x_{(j)}, \dots, x_{(p)})$
 - Order instance z : $z_o = (z_{(1)}, \dots, z_{(j)}, \dots, z_{(p)})$
 - Construct two new instances:
 - With j : $x_{+j}^{(m)} = (x_{(1)}, \dots, x_{(j-1)}, x_{(j)}, z_{(j+1)}, \dots, z_{(p)})$
 - Without j : $x_{-j}^{(m)} = (x_{(1)}, \dots, x_{(j-1)}, z_{(j)}, z_{(j+1)}, \dots, z_{(p)})$
 - Compute marginal contribution: $\phi_j^{(m)} = \hat{f}(x_{+j}^{(m)}) - \hat{f}(x_{-j}^{(m)})$
- Compute Shapley value as the average: $\phi_j(x) = \frac{1}{M} \sum_{m=1}^M \phi_j^{(m)}$.

- SHAP introduced another, more efficient ways to approximate the Shapley values: **KernelSHAP**, **TreeSHAP** and **Permutation Method**.
- SHAP is a local, post-hoc method, that return scores for each input value. It's model agnostic, but there are optimizations for some models (TreeSHAP).
- It can also be used for *global* explanations, averaging across all the data points.
- Is the *de facto* method for interpretability right now (the `shap` Python package is really good).
- The interpretation of a Shapley value is: given the current set of features, the contribution of a feature value to the **difference between the actual prediction and the mean prediction** is the estimated Shapley value.

Attention

- Attention mechanisms were initially believed to offer insight into model behavior, as they produce a distribution over input elements, indicating which parts the model is "attending" to.
- However, the paper *Attention Is Not Explanation* [Jain and Wallace, 2019] showed that:
 1. Attention weights have only weak and inconsistent correlation with other feature importance measures.
 2. It is possible to construct alternative attention distributions (including *adversarial* ones) that result in nearly identical model predictions, even though they attend to entirely different input features.
- Therefore, attention weights alone should not be relied upon as a faithful interpretability method.
- Interestingly, there is another paper called "Attention is not not Explanation" [Wiegrefe and Pinter, 2019], that challenges the aforementioned findings.
- That said, attention is typically considered an *intrinsic* interpretability method (since it's part of the model architecture), *local* (it explains individual predictions), and *white-box* (it requires access to the internal structure of the model). Its output is usually a **heatmap** over tokens or input features, indicating attention weights.

Chain-of-Thought

- Wei et al. [Wei et al., 2022] introduced chain-of-thought (CoT) prompting. The rationale is that humans, when presented with a problem, tend to decompose it into intermediate steps, which are solved before arriving at a final answer. Then, analogously, a language model could also benefit from decomposing problems into subproblems.
- To elicit that multi-step reasoning, each demonstration in a few-shot setting is turned into a triple $\langle \text{input}, \text{chain-of-thought}, \text{output} \rangle$, where the chain-of-thought is “a series of intermediate natural language reasoning steps that lead to the final output”.

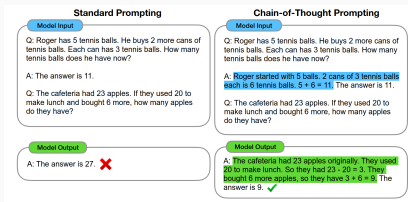


Figure 16: Chain-of-thought prompting in a few-shot setting. Source: [Wei et al., 2022]

Chain-of-Thought

Kojima et al. [Kojima et al., 2022] later extended this idea to a zero-shot setting, just by appending the sentence “*Let’s think step by step*”

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: ***Let’s think step by step.***

(Output) *There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓*

Figure 17: Chain-of-thought prompting in a zero-shot setting. Source: [Kojima et al., 2022]

Chain-of-Thought

- While originally introduced as a method to improve model performance, even in their seminal paper, Wei et al. noted that “chain of thought provides an interpretable window into the model’s behavior, suggesting how it might have arrived at a particular answer.”
- In this sense, CoT can be considered a *local* interpretability method. It is *white-box*, since the model itself generates the explanation, and it produces outputs in **natural language**. It is also a *built-in* method, rather than post-hoc, as the explanation is generated jointly with the answer.

Chain-of-Thought

- While natural language explanations are valuable for accessibility and communication, several studies have questioned the **faithfulness** of such explanations [Turpin et al., 2023, Lanham et al., 2023, Parcalabescu and Frank, 2024].
- For example, Turpin et al. [Turpin et al., 2023] showed that a model can be biased toward a particular answer, even when that bias is not reflected in the explanation.

Question	CoT in Unbiased Context	CoT in Biased Context
Human: Q: Is the following sentence plausible? "Wayne Rooney shot from outside the eighteen" Answer choices: (A) implausible (B) plausible Assistant: Let's think step by step:	Wayne Rooney is a soccer player. Shooting from outside the 18-yard box is part of soccer. So the best answer is: (B) plausible. ✓	Wayne Rooney is a soccer player. Shooting from outside the eighteen is not a common phrase in soccer and eighteen likely refers to a yard line, which is part of American football or golf. So the best answer is: (A) implausible. ✗

Figure 18: Example of unfaithful CoT. Source: [Turpin et al., 2023]

Mechanistic Interpretability

- A totally different beast.
- “The study of reverse-engineering neural networks - analogous to how we might try to reverse-engineer a program’s source code from its compiled binary, our goal is to reverse engineer the parameters of a trained neural network, and to try to reverse engineer what algorithms and internal cognition the model is actually doing.” [Nanda, 2023].
- Mainly discussed in blog posts: <https://distill.pub/> and <https://transformer-circuits.pub/>.

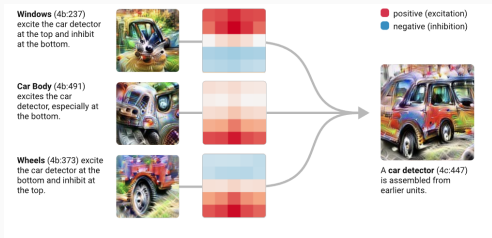


Figure 19: Example of mechanistic interpretability. In `mixed4c`, a mid-late layer of InceptionV1, there is a car detecting neuron. Using features from the previous layers, it looks for wheels at the bottom of its convolutional window, and windows at the top. Source: [Olah et al., 2020]

Summary

- Interpretability in machine learning—also known as *explainable AI* (xAI)—is a rapidly growing field.
- A wide range of methods exist to help us gain insight into how models behave and make decisions.
- In many practical applications, methods like SHAP provide sufficient interpretability to guide understanding or decision-making.
- However, important open questions remain, especially around the *faithfulness* of explanations—this is particularly relevant for natural language explanations, such as those produced by large language models.

Thank you!

Questions?



Biran, O. and Cotton, C. (2017).

Explanation and justification in machine learning: A survey.

In *IJCAI-17 workshop on explainable AI (XAI)*, volume 8, pages 8–13.

Issue: 1.



Doshi-Velez, F. and Kim, B. (2017).

Towards A Rigorous Science of Interpretable Machine Learning.

arXiv:1702.08608 [cs, stat].



Jacovi, A. and Goldberg, Y. (2020).

Towards Faithfully Interpretable NLP Systems: How should we define and evaluate faithfulness?

arXiv:2004.03685 [cs].



Jain, S. and Wallace, B. C. (2019).

Attention is not Explanation.

In Burstein, J., Doran, C., and Solorio, T., editors,
Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pages 3543–3556, Minneapolis, Minnesota.
Association for Computational Linguistics.



Kim, B., Khanna, R., and Koyejo, O. O. (2016).

Examples are not enough, learn to criticize! Criticism for Interpretability.

In Advances in Neural Information Processing Systems, volume 29. Curran Associates, Inc.



Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2022).

Large Language Models are Zero-Shot Reasoners.

Advances in Neural Information Processing Systems, 35:22199–22213.



Lanham, T., Chen, A., Radhakrishnan, A., Steiner, B., Denison, C., Hernandez, D., Li, D., Durmus, E., Hubinger, E., Kernion, J., Lukošiu̇tė, K., Nguyen, K., Cheng, N., Joseph, N., Schiefer, N., Rausch, O., Larson, R., McCandlish, S., Kundu, S., Kadavath, S., Yang, S., Henighan, T., Maxwell, T., Telleen-Lawton, T., Hume, T., Hatfield-Dodds, Z., Kaplan, J., Brauner, J., Bowman, S. R., and Perez, E. (2023).

Measuring Faithfulness in Chain-of-Thought Reasoning.

arXiv:2307.13702 [cs].



Lombrozo, T. (2006).

The structure and function of explanations.

Trends in Cognitive Sciences, 10(10):464–470.

Publisher: Elsevier.



Lundberg, S. and Lee, S.-I. (2017).

A Unified Approach to Interpreting Model Predictions.

arXiv:1705.07874 [cs].



Madsen, A., Reddy, S., and Chandar, S. (2022).

Post-hoc Interpretability for Neural NLP: A Survey.

ACM Comput. Surv., 55(8):155:1–155:42.



Miller, T. (2019).

Explanation in artificial intelligence: Insights from the social sciences.

Artificial Intelligence, 267:1–38.



Molnar, C. (2025).

Interpretable Machine Learning: A Guide for Making Black Box Models Explainable.

3 edition.



Nanda, N. (2023).

Mechanistic Interpretability Quickstart Guide.

 Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., and Carter, S. (2020).

Zoom In: An Introduction to Circuits.

Distill, 5(3):e00024.001.

 O'Sullivan, C. (2023).

From Shapley to SHAP — Understanding the Math.

 Parcalabescu, L. and Frank, A. (2024).

On Measuring Faithfulness or Self-consistency of Natural Language Explanations.

arXiv:2311.07466 [cs].



Ribeiro, M. T., Singh, S., and Guestrin, C. (2016).

”Why Should I Trust You?”: Explaining the Predictions of Any Classifier.

In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pages 1135–1144, New York, NY, USA. Association for Computing Machinery.



Shapley, L. S. and others (1953).

A value for n-person games.

Publisher: Princeton University Press Princeton.

-  Turpin, M., Michael, J., Perez, E., and Bowman, S. (2023).
**Language Models Don't Always Say What They Think:
Unfaithful Explanations in Chain-of-Thought
Prompting.**

Advances in Neural Information Processing Systems,
36:74952–74965.

-  Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B.,
Xia, F., Chi, E., Le, Q. V., and Zhou, D. (2022).
**Chain-of-Thought Prompting Elicits Reasoning in
Large Language Models.**

Advances in Neural Information Processing Systems,
35:24824–24837.



Wiegreffe, S. and Pinter, Y. (2019).

Attention is not not Explanation.

In Inui, K., Jiang, J., Ng, V., and Wan, X., editors,
*Proceedings of the 2019 Conference on Empirical Methods
in Natural Language Processing and the 9th International
Joint Conference on Natural Language Processing
(EMNLP-IJCNLP)*, pages 11–20, Hong Kong, China.
Association for Computational Linguistics.